



Algorithms and Complexity 2026

Exercise Session 3: Dynamic Programming

This is a collection of problems that hopefully improve your understanding of the material discussed in the lectures. During the exercise classes you have time to work on the first 3-5 problems and will have the opportunity to obtain feedback and/or help from one of the teaching assistants. Towards the end of each session the solutions of these problems will be covered.

To optimally prepare for the mastery test, you are encouraged to solve some of the remaining problems on your own. If you are stuck, require assistance or someone really messed up to write an understandable solution, feel free to send an email to algokomp-help@kth.se or visit an office hour.

Some problems are marked with superscripts: 1^M , 2^* or even 3^{**} . Problems marked with an **M** superscript are in the spirit of mastery test problems. Such problems are usually not stated as plain math problems but rather consist of a story from which you need to extract the mathematical structure yourself. Problems marked with a $\star$ are challenging problems. The more stars the harder – do not get discouraged if you cannot solve such a problem.

1 Multiplication

A simple procedure to multiply two number a and b proceeds as follows. Multiply a times the least significant digit of b and record this as the first intermediate result. Then, proceed by multiplying a times the second least significant digit of b and note the result as the second intermediate result. Continue until all digits of b have been multiplied with a . Last, add up all the intermediate results with the correct “shift” to the left. For an example consult Figure 1.

$$\begin{array}{r}
 37 \times 4831 \\
 \hline
 37 = 1 \times 37 \\
 + 111 = 3 \times 37 \\
 + 296 = 8 \times 37 \\
 + 148 = 4 \times 37 \\
 \hline
 = 178747
 \end{array}$$

Figure 1: Multiplication of 37 with 4831. The intermediate results are written beneath the original multiplication with the appropriate shift. These are then added up to the final result of 178 747.

[Algorithm 1](#) attempts to formalize above multiplication procedure. It relies on the ADD procedure from the previous assignment to add up two n -digit numbers.

The algorithm takes an m -digit number $a = a_{m-1}a_{m-2} \cdots a_0$ and a n -digit number $b = b_{n-1}b_{n-2} \cdots b_0$ as input. To reduce clutter, we assume that $m \leq n$ at the beginning of the

algorithm. Note that when we do a recursive call we are able to ensure this condition. Hence this is not a significant simplification.

Algorithm 1 multiplies the m - and n -digit numbers a and b .

```

1: function MULTIPLY( $a, b, m, n$ )                                ▷ We assume that  $1 \leq m \leq n$ 
2:   if  $n$  is 1 then
3:     return  $a_0 \cdot b_0$ 
4:    $r \leftarrow 00 \cdots 0$ 
5:   for  $i \leftarrow 0, 1, \dots, n-1$  do
6:      $t \leftarrow \text{MULTIPLY}(b_i, a, 1, m)$ 
7:      $r \leftarrow \text{ADD}(r_{m+i}r_{m+i-1} \cdots r_0, t_mt_{m-1} \cdots t_0 00 \cdots 0, m+i+1)$ 
8:   return  $r$ 

```

- (a) In the context of this algorithmic problem (multiplication of two integers) and [Algorithm 1](#), are we operating in a bit cost or unit cost model?
- (b) What is the maximum value of any a_i or b_i ?
- (c) What is the time complexity if $m = 1$?
- (d) Using the result of the previous question, can you analyze the time complexity of the algorithm if m and n take any value?
- (e) Can you improve the time complexity by modifying the call to ADD?
- (f) This algorithm is often presented as a non-recursive algorithm. Try to rewrite [Algorithm 1](#) as an iterative algorithm using two for loops.

Solution:

- (a) In a unit cost model we would just assume that multiplication of two integers takes $O(1)$ time and be done, the problem only gets interesting when we consider how to multiply arbitrarily large numbers by operating on the individual digits of the numbers (as [Algorithm 1](#) does). In other words we are working in a bit cost model.
- (b) a_i and b_i are digits and hence in $\{0, 1, \dots, 9\}$.
- (c) $O(n^2)$, as the call to ADD is of order $O(n)$ for the last $n/2$ iterations of the for loop.
- (d) The for loop is executed n times. Note that each iteration takes time $O(m^2 + n)$ (from the previous question) and hence the overall time complexity is $O(n(m^2 + n))$.
- (e) Note that when invoking ADD we are not interested in the i least significant digits: these are equal to $r_{i-1} \cdots r_0$, as the second input is 0 in these digits. Hence we can leave the i least significant bits of r be and only change the $m+1$ most significant bits: we invoke $\text{ADD}(r_{m+i}r_{m+i-1} \cdots r_i, t, m+1)$.

This improves the time complexity in (c) to $O(n)$, as now the addition is a constant operation. This in turn leads to an overall time complexity of $O(n \cdot m)$.

(f) See [Algorithm 2](#).

Algorithm 2 multiplies the m - and n -digit numbers a and b .

```
1: function MULTIPLY( $a, b, m, n$ )
2:    $r \leftarrow 00 \dots 0$ 
3:   for  $i \leftarrow 0, 1, \dots, m - 1$  do
4:      $c_0 \leftarrow 0$ 
5:     for  $j \leftarrow 0, 1, \dots, n - 1$  do
6:        $t \leftarrow r_{i+j} + a_i b_j + c_j$ 
7:        $r_{i+j} \leftarrow t \bmod 10$ 
8:        $c_{j+1} \leftarrow (t - r_{i+j}) / 10$ 
9:      $r_{i+n} \leftarrow c_n$ 
10:  return  $r$ 
```

2 Longest Common Substring

The string **ALGORITHM** and the string **PLÅGORIS** have the common substring **GORI**. The *longest common substring* of these strings has thus length 4. The letters in a substring must form a coherent sequence.

Construct an efficient algorithm that given two strings $a_1 a_2 \dots a_m$ and $b_1 b_2 \dots b_n$ calculates the length of the longest common substring. The algorithm uses dynamic programming and runs in time $O(nm)$.

Solution:

Let $M[i, j]$ be the number of letters to the left of (and including) a_i which comply with the same number of letters to the left (and including) b_j . The length of the longest common string is the largest number in matrix M .

M can be defined recursively as follows:

$$M[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ M[i - 1, j - 1] + 1 & \text{if } a_i = b_j, \\ 0 & \text{otherwise.} \end{cases}$$

The following algorithm computes M and returns the largest number in M .

Algorithm 3 returns the length of the longest common substring of a and b .

```
1: function LONGESTSUBSTRING( $a, b, m, n$ )
2:    $M \leftarrow m \times n$  matrix
3:    $\max \leftarrow 0$ 
4:   for  $j \leftarrow 0, 1, \dots, n$  do
5:      $M[0, j] \leftarrow 0$ 
6:   for  $i \leftarrow 1, 2, \dots, m$  do
7:      $M[i, 0] \leftarrow 0$ 
8:     for  $j \leftarrow 1, 2, \dots, n$  do
9:       if  $a_i = b_j$  then
10:         $M[i, j] \leftarrow M[i - 1, j - 1] + 1$ 
11:        if  $M[i, j] > \max$  then
12:           $\max \leftarrow M[i, j]$ 
13:       else
14:          $M[i, j] \leftarrow 0$ 
15:   return  $\max$ 
```

The running time is dominated by the nested *for* loops and therefore is $\Theta(nm)$.

3 Checkerboard

Suppose we are given an $n \times n$ checkerboard and a checker. We have to move the checker from the bottom edge to the top edge of the board. At each step we may move the checker to one of the following three squares:

- the square immediately above,
- the square that is one up and one to the left, or
- the square that is one up and one to the right.

Along with the checkerboard we are also given a payoff function p : given two squares s_1, s_2 that constitute a legal move, $p(s_1, s_2)$ is the cost (or gain) of this move.

Devise an algorithm that moves a checker from the bottom edge to the top edge of the board while maximizing the payoff. Your algorithm is free to pick any square along the bottom edge as a starting point and any square along the top edge as the final destination. Do not forget to analyze the time complexity of your algorithm.

Solution:

Let us use Dynamic Programming to solve this problem. Denote by $M(i, j)$ the maximum payoff that can be achieved from square (i, j) to the top edge of the board (the squares $(1, \ell)$, for $\ell = 1, 2, \dots, n$).

To express $M(i, j)$ as a recursive formula, let us define shorthands for the payoff if we move up-left, straight up or up-right:

$$\begin{aligned} L_{i,j} &= M(i - 1, j - 1) + p((i, j), (i - 1, j - 1)), \\ A_{i,j} &= M(i - 1, j) + p((i, j), (i - 1, j)), \\ R_{i,j} &= M(i - 1, j + 1) + p((i, j), (i - 1, j + 1)). \end{aligned}$$

We can then express $M(i, j)$ as as the maximum of above quantities that are valid;

$$M(i, j) = \begin{cases} 0, & \text{if } i = 1 \\ \max\{A_{i,j}, R_{i,j}\}, & \text{if } j = 1 \\ \max\{L_{i,j}, A_{i,j}\}, & \text{if } j = n \\ \max\{L_{i,j}, A_{i,j}, R_{i,j}\}, & \text{otherwise.} \end{cases} \quad (1)$$

Note that the maximum payoff is one of $M(n, \ell)$, for $\ell = 1, 2, \dots, n$.

All that is left to do is to explain how we can compute M . [Algorithm 4](#) shows how to compute M efficiently.

Algorithm 4

```

1: function CHECKER( $p, n$ )
2:    $M \leftarrow n \times n$  matrix
3:   for  $\ell \leftarrow 1, 2, \dots, n$  do ▷ initialize the top edge to 0
4:      $M[1, \ell] \leftarrow 0$ 
5:   for  $i \leftarrow 2, 3, \dots, n$  do
6:      $M[i, 1] \leftarrow \max\{A_{i,1}, R_{i,1}\}$ 
7:     for  $j \leftarrow 2, 3, \dots, n-1$  do
8:        $M[i, j] \leftarrow \max\{L_{i,j}, R_{i,j}, A_{i,j}\}$ 
9:      $M[i, n] \leftarrow \max\{L_{i,n}, A_{i,n}\}$ 
10:  return  $\max_{1 \leq \ell \leq n} M(n, \ell)$ 

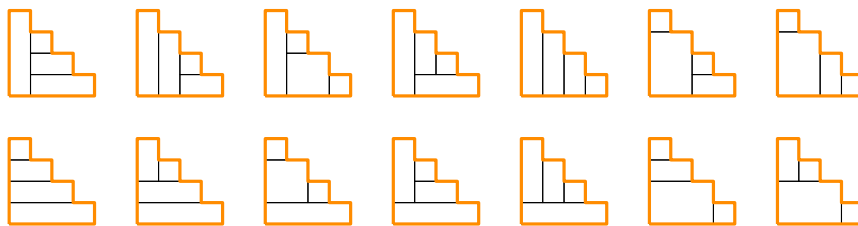
```

Time Complexity Note that the cost of computing a single entry in M is constant and hence the time complexity is $\mathcal{O}(n^2)$, as M is an $n \times n$ matrix.

4 Tiling Stairs

Let A_n be the number of ways a stair shape of height n can be tiled by n rectangles.

Example: The following figure shows all possible tilings for $n = 4$. Hence $A_4 = 14$.



- Come up with a recurrence relation for A_n . In other words, express A_n as a function of $A_0, A_1, \dots, A_{n-1}$.
- Using this recurrence relation, devise an algorithm to compute the number A_n . What is the time complexity of this algorithm?

Hints: Below are some hints to help you come up with the recurrence relation.

Hint 1: „Observe that the ‘steps’, highlighted below, must be in distinct rectangles of the tiling“



Hint 2: Let us denote by R_i the rectangle that contains step i for $i \in \{1, 2, \dots, n\}$. Observe that exactly one of these will contain the bottom left corner of the stairs, as highlighted below.



Hint 3: After removing the rectangle from the previous hint you are left with two smaller stairs. Use this to come up with a recurrence relation.

Solution:

- (a) Pick the “step” $i \in \{1, 2, \dots, n\}$ that contains the bottom left corner. Note that after fixing this rectangle, we are left with two smaller stairs of size $i - 1$ and size $n - i$. Each of these can be tiled in A_{i-1} and A_{n-i} ways independently. Thus we obtain the recurrence relation

$$A_n = \sum_{\ell=1}^n A_{\ell-1} \cdot A_{n-\ell}, \quad (2)$$

with base cases $A_0 = 1$ and $A_1 = 1$. This sequence is known as the Catalan Numbers and occurs in various counting problems. See [Wikipedia](#) for more information.

- (b) In order to compute A_n , we need to know the value of $A_0, A_1, \dots, A_{n-1}$. Thus we recursively compute these values but memorize them. I.e., we compute the value of A_i once and then store it so the next time we need A_i we can just look it up. [Algorithm 5](#) attempts to formalize this idea.

Algorithm 5 returns the number of ways a stair shape of height n can be tiled by n rectangles.

```

1: function STAIRTILINGS( $n$ )
2:    $A \leftarrow$  array of length  $n + 1$ 
3:    $A[0], A[1] \leftarrow 1$  ▷ Base Cases
4:   for  $i \leftarrow 2, 3, \dots, n$  do
5:      $s \leftarrow 0$ 
6:     for  $\ell \leftarrow 1, 2, \dots, i$  do
7:        $s \leftarrow s + A[\ell - 1] \cdot A[i - \ell]$ 
8:      $A[i] \leftarrow s$ 
9:   return  $A[n]$ 

```

Time Complexity There are two for-loops. The outer loop is executed n times and the inner loop is iterated at least $n/2$ times for the last $n/2$ iterations of the outer loop.

In the unit-cost model, the inner most loop does constant work and hence the time complexity is $\mathcal{O}(n^2)$.

But is the unit-cost model the right model to use? Note that $A_n \geq 2 \cdot A_{n-1}$ and thus A_n requires at least n bits to be represented. Thus the inner most loop in fact does non-constant work in the bit-cost model. In order to get an upper bound on the amount of work done, we need an upper bound on the number of bits A_n may consist of.

This is not so easy to derive but the [Wikipedia](#) page states that $A_n \in \mathcal{O}(4^n)$. Hence in the bit-cost model we see that the inner loop in fact takes time $\mathcal{O}(n^2)$ (assuming this is the cost of multiplying two n -bit numbers) and hence, in the bit-cost model, the algorithm has time complexity $\mathcal{O}(n^4)$.

5 Sequences

You are given two sequences of positive integers $a_1, a_2, \dots, a_n$ and $b_1, b_2, \dots, b_n$, where all numbers are less than n^2 , and a positive integer B , such that $B \leq n^3$. The problem is to determine if there is a sequence $c_1, c_2, \dots, c_n$ such that $\sum_{i=1}^n c_i = B$ and $c_i = a_i$ or $c_i = b_i$ for $1 \leq i \leq n$.

Describe and analyze an algorithm that solves this problem by using dynamic programming. Moreover, describe how to extend the algorithm to construct the solution, where $c_i = a_i$ or $c_i = b_i$ for $1 \leq i \leq n$.

Solution:

We create a Boolean $n \times B$ -matrix M that is initially contains only zeros. A one in $M[k, s]$ means that there is a choice of $c_1, \dots, c_k$ such that $\sum_{i=1}^k c_i = s$.

The recursive equation for $M[k, s]$ becomes:

$$M[k, s] = \begin{cases} 1 & \text{if } k = 1 \text{ and } (s = a_1 \text{ or } s = b_1), \\ 1 & \text{if } k > 1 \text{ and } (M[k-1, s-a_k] = 1 \text{ or } M[k-1, s-b_k] = 1), \\ 0 & \text{otherwise.} \end{cases}$$

The calculation begins with $M[1, a_1]$ and $M[1, b_1]$ set to 1. Then we put ones in $M[2, a_1 + a_2]$, $M[2, a_1 + b_2]$, $M[2, b_1 + a_2]$ and $M[2, b_1 + b_2]$ in the second row of the matrix, then we put ones in the third row and so on. If there is a one in $M[n, B]$, the answer to the problem is yes. The following algorithm formalizes this procedure.

Algorithm 6 returns 1 iff there is a sequence $c_1, \dots, c_n$ as required.

```

1: function EXISTSC( $a, b, n, B$ )
2:    $M \leftarrow n \times B$  matrix, initialized to 0
3:    $M[1, a_1], M[1, b_1] \leftarrow 1$  ▷ Base case
4:   for  $k \leftarrow 2, 3, \dots, n$  do
5:     for  $s \leftarrow 1, 2, \dots, B$  do
6:       if  $M[k-1, s-a_k] = 1$  or  $M[k-1, s-b_k] = 1$  then ▷ Assuming no index < 0
7:          $M[k, s] \leftarrow 1$ 
8:   return  $M[n, B]$ 

```

The nested *for* loop runs $(n-1)B$ times and each loop performs a constant number of operations. The whole running time of the algorithm is therefore in $\mathcal{O}(nB + (n-1)B) = \mathcal{O}(n^4)$. You can construct the solution by starting from the end (from the n -th row and up) and looking for positions where there is a one. The one in $M[n, B]$ must be there because of a one in either $M[n-1, B-a_n]$ or $M[n-1, B-b_n]$. Remember which of these it is. If a one occurs in two positions, just pick one (as only one solution is requested) and continue in this way from the selected position. More formally the algorithm can be described as follows.

Algorithm 7 returns a sequence $c_1, \dots, c_n$ as required. M is the matrix from [Algorithm 6](#).

```
1: function WRITEC( $a, b, M, n, B$ )
2:    $s \leftarrow B$ 
3:   for  $i \leftarrow n, n-1, \dots, 2$  do
4:     if  $M[i-1, s-a_i] = 1$  then
5:        $c_i \leftarrow a_i$ 
6:     else
7:        $c_i \leftarrow b_i$ 
8:      $s \leftarrow s - c_i$ 
9:    $c_1 \leftarrow s$ 
10:  return  $c_1, \dots, c_n$ 
```
